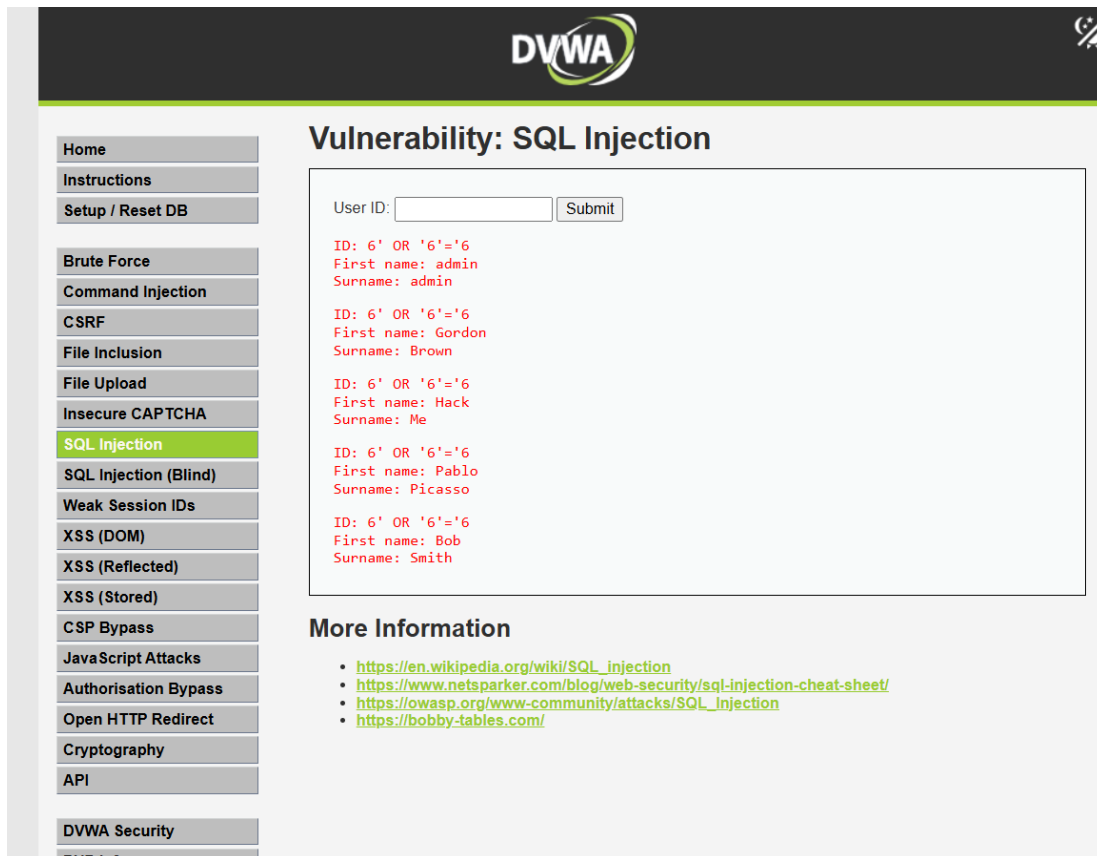


Week 3 Web Security Basics: DVWA Practice

1. SQL Injection



What Happened

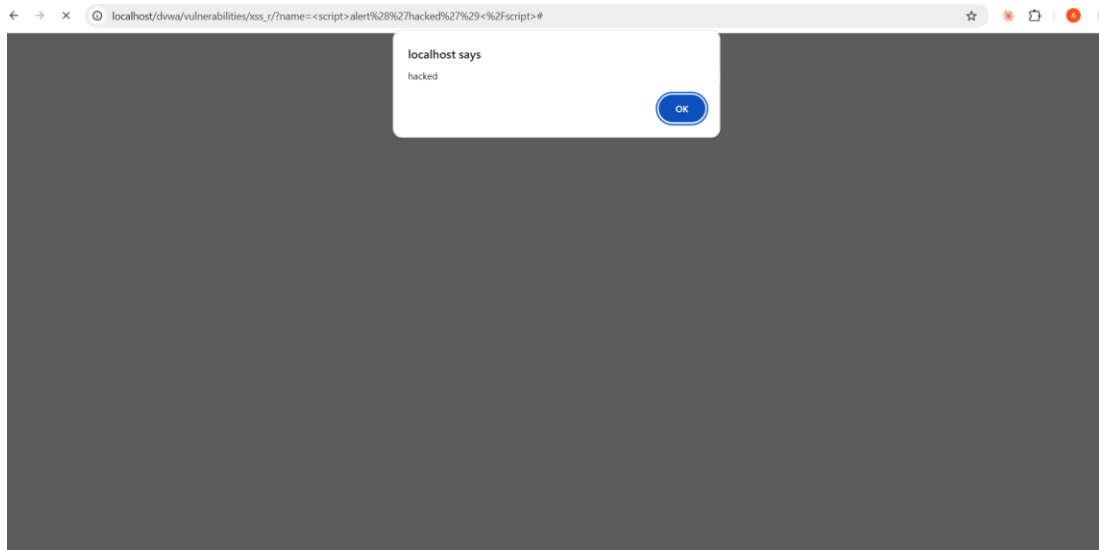
- The input field does not filter user data. There are no checks or conditions to verify whether the input is harmful or harmless.
- Because of this, an attacker can type a SQL trick (like adding a quote and an always-true condition) instead of normal data, and the database accepts it as a valid command.
- The database query directly takes the user input and combines it with the query. It does not separate user input from the actual command, so whatever is typed becomes part of the query itself.
- Since there are no restrictions, the query returns all the data from the database instead of just one record.

How to Prevent It

- Developers must validate and filter user input before it reaches the database (checking whether it contains unexpected characters like quotes).

- The database query should use prepared statements / parameterized queries, so user input is treated as data only, not as part of the command.
- Limit the database user's permissions so even if an injection happens, the damage is limited.
- Show generic error messages to users instead of detailed database errors, so attackers get no extra information.

2. XSS (Reflected)



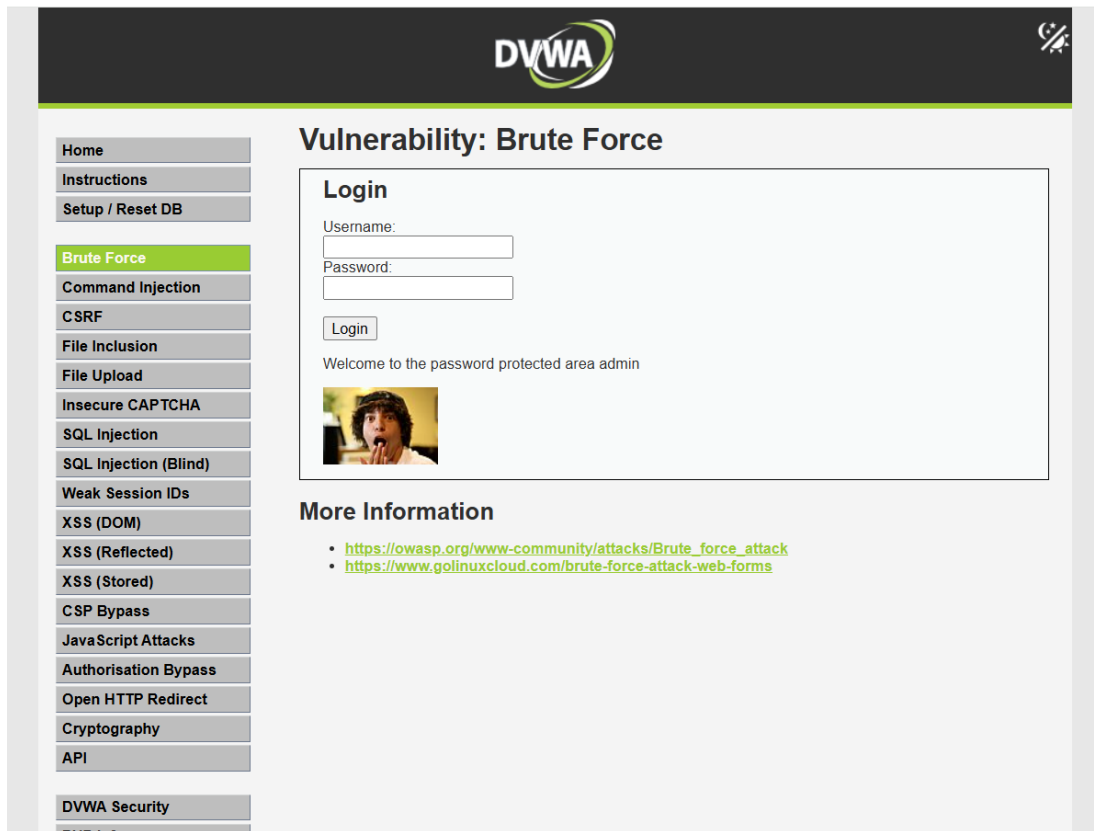
What Happened

- There is no filter on the user input field. It accepts whatever is typed without any checks or conditions.
- The input is directly embedded into the HTML page without verification.
- Because of this, harmful data (like a script tag) gets treated as actual code instead of plain text.
- The page shows the same output that was typed, including the harmful script, which then executes in the browser (the alert popup is proof of this).

How to Prevent It

- Filter and validate user input before accepting it.
- Do not embed user input directly into the HTML page without checking/sanitizing it first.
- Use functions like htmlspecialchars() (in PHP) to convert special characters into safe text so scripts cannot execute.
- Apply a Content Security Policy (CSP) to restrict which scripts are allowed to run on the page.

3. Brute Force



What Happened

- There is no login attempt limit, so the same username and password can be tried unlimited times without any restriction.
- There is no two-factor (two-way) verification step, so only a username and password are enough to log in.
- There is no lockout mechanism, so the system does not stop or slow down repeated login attempts, which allows an attacker to try many passwords continuously (manually or with a tool).

How to Prevent It

- Add a login attempt limit (e.g., lock the account after 3-5 failed attempts).
- Enable two-factor authentication (2FA) so a password alone is not enough to log in.
- Add an account lockout or time delay after repeated failed attempts to slow down or stop attackers.
- Use CAPTCHA to block automated tools from trying passwords continuously.
- Enforce strong password policies so passwords are harder to guess.